

DeployGuard

Technical Architecture, Security Contracts & Competitive Analysis

AWS-native PaaS control plane built around immutable release evidence

GitHub App + OIDC | Railpack + certified fallback | Terraform + ECS Fargate

Immutable rollback | Managed DB + EFS | Fail-closed verification

Certification snapshot - 17 September 2026

Source/static contract closure: PASS

Newest immutable-pin AWS E2E: UNVERIFIED / externally blocked before dispatch

Hassan Sajjad | Final Year Project | Air University Islamabad

Executive Summary

What DeployGuard is, what is implemented, and what is actually verified today

DeployGuard is an AWS-oriented Platform-as-a-Service control plane that turns a GitHub repository into a controlled release on ECS/Fargate. Its distinguishing idea is not merely “build and run a container”; it binds source identity, build interpretation, configuration, cloud permissions, infrastructure state, immutable image identity, runtime health and terminal evidence into one release contract. The platform is built around fail-closed promotion: a workflow success is not sufficient by itself - the backend accepts a release only after identity-bound evidence is validated.

Current certification status

Test 2 head 54ddc957... pins df86ae471...; Test 3 head 1d5a243a... pins 0ebf940d.... Static/source contract closure is PASS on both branches. The newest immutable pins have NOT yet completed AWS end-to-end certification because execution was blocked before dispatch by backend authentication/credential configuration and the absence of a separately configured Test 3 AWS account. This document therefore distinguishes implemented controls from live-certified behavior.

Dimension	Current assessment
Architecture / source implementation	Implemented across frontend, NestJS control plane, GitHub workflow, Terraform runtime and evidence validators.
Static / contract testing	PASS for the final defect-closure set: build, dynamic PORT resolution, environment-scoped state simulation, Railpack runtime contract, action pins and release identity.
Historical Test 2 AWS evidence	Normal Deploy, Rollback and Destroy were executed successfully on an earlier pin; changed-source redeploy and managed-DB lifecycle were not completely certified then.
Newest-pin AWS E2E	UNVERIFIED / BLOCKED before dispatch. No new runtime failure has been observed on the final pins.
Test 3	Source/static checks pass; live certification still requires a correct separately configured AWS environment.

Product thesis

- **Developer experience:** GitHub-connected deployment with automatic service/build/port/config admission and lifecycle controls from one product UI.
- **Infrastructure ownership:** The target AWS account owns ECR, ECS, ALB, security groups, EFS, Secrets Manager, Cloud Map and Terraform state rather than hiding all infrastructure behind a provider-only abstraction.
- **Release truth:** A stable release is an immutable identity bundle - source SHA, configuration/build revisions, ECR digest, task definition and verified runtime evidence.
- **Safety:** Destructive actions, trust mutation, secret handling and rollback are bounded by explicit ownership, namespace and identity checks.

Document Map

A technical walkthrough from product surface to AWS runtime

Section	Topic
1	Product scope & user journey
2	System architecture & trust boundaries
3	GitHub integration & repository admission
4	Build-target, port & configuration contracts
5	Railpack build and certified Docker fallback
6	Immutable image and release identity
7	Terraform/AWS runtime topology
8	Deploy, redeploy, rollback and destroy semantics
9	Managed database architecture
10	State, concurrency and recovery
11	Security architecture - 24 controls
12	Observability, diagnostics, cost, billing, notifications and AI
13	Performance behavior and scaling model
14	Verification status and certification gates
15	Vercel vs Render vs Heroku - clear contract differences
16	Limitations, hardening roadmap and demo sequence
A	Source references and external documentation

Reading convention

"Implemented" means a concrete source/runtime contract exists. "Statically verified" means repository checks cover the invariant. "AWS E2E verified" is reserved for a real dispatched run on the exact current pins. This distinction prevents the report from turning static confidence into an unsupported production claim.

1. Product Scope & User Journey *[DG-01, DG-02]*

The product surface is broader than a single deploy button. The NestJS application registers authenticated project management, GitHub connectivity, deployable services, environment variables, managed database settings, deployment history/current state, rollback and destroy controls, observability, Terraform export, AI troubleshooting, billing and administrative/audit capabilities. The React/Vite frontend has dedicated contract tests for the deployment journey, managed environment, recovery, monitoring, billing/Trivy presentation and lifecycle state synchronization.

1.1 Primary user journey

1. Authenticate through GitHub OAuth and establish the DeployGuard application session.
2. Connect a GitHub App installation and select a repository, branch/exact source and deployable service directory.
3. Configure service-level runtime/build variables and, optionally, attach one managed database tier.
4. Deploy. The backend freezes an immutable admission snapshot and dispatches a GitHub Actions caller pinned to the certified reusable workflow.
5. Observe build, infrastructure and runtime stages through project current state, history, logs/metrics and structured failure diagnostics.
6. Redeploy changed source through the direct ECS release path when topology is unchanged.
7. Rollback to the immediate eligible stable release without rebuilding historical source.
8. Destroy with explicit confirmation; finalize the project only after cloud cleanup evidence passes.

1.2 Role model and protected surfaces

All API routes receive the authentication middleware. Projects are guarded for ADMIN, DEVELOPER and READONLY access, while mutating service/lifecycle operations are narrowed to ADMIN or DEVELOPER. Administrator authentication is intentionally separated from GitHub developer sessions. [DG-03, DG-04]

Role / identity	Typical authority	Boundary
Developer GitHub session	Project read/write, deploy/redeploy/rollback/destroy where allowed	Signed session audience=developer; GitHub-linked user; mutating endpoints require developer/admin role.
Readonly user	Project and state visibility	No mutating deployment/service operations.
Admin session	Administrative functions	Separate admin audience and login path; GitHub-linked admin sessions are rejected.
Backend AWS identity	Admission, account checks, OIDC/IAM reconciliation, runtime-secret materialization, managed-DB reconciliation	Separate from the GitHub Actions execution role.
GitHub Actions role	AWS deployment execution	Assumed through GitHub OIDC for the exact repository trust context.

2. System Architecture & Trust Boundaries *[DG-02, DG-05]*

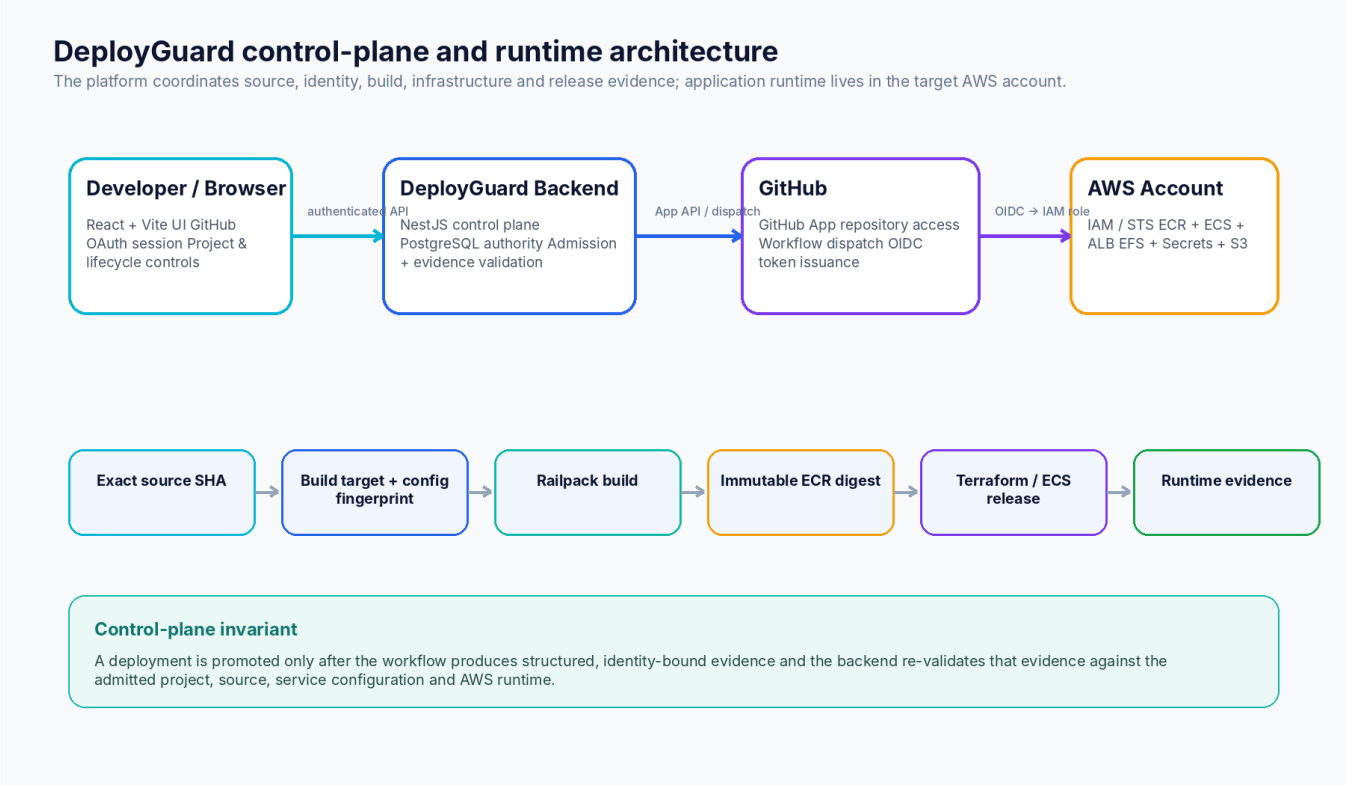


Figure 1. Control-plane and runtime ownership model.

2.1 Control plane

- **Frontend:** React 18 + Vite, project dashboard and lifecycle UX; browser-facing auth is session-cookie based.
- **Backend:** NestJS 11 + TypeORM + PostgreSQL. It is the authority for projects, immutable configuration/build revisions, pipeline runs, stable releases, audit events, database binding state, observability projections and billing records.
- **GitHub plane:** GitHub OAuth authenticates developers; GitHub App installation credentials read repositories and dispatch the per-repository caller workflow.
- **Deployment plane:** The reusable workflow checks out an exact application SHA, assumes AWS through OIDC, builds images, materializes Terraform from the exact certified control-plane SHA, runs lifecycle operations and publishes evidence.
- **AWS runtime:** ECR, ECS/Fargate, ALB/target groups/listeners, security groups, CloudWatch Logs, IAM execution roles, S3 Terraform state and optional Secrets/EFS/Cloud Map managed-database resources.

2.2 Two AWS identities by design

DeployGuard does not treat “AWS credentials” as one undifferentiated permission set. The backend identity performs pre-dispatch platform operations and reads/reconciles existing state. GitHub Actions assumes a separate execution role through OIDC for Terraform/build/runtime operations. The backend verifies that its STS caller account, configured AWS account ID and the account embedded in the Actions role ARN all agree before cloud dispatch. This prevents a valid role from silently targeting the wrong AWS account.

2.3 Shared foundation vs project-owned resources

Shared / external foundation	Project-owned / lifecycle-managed
AWS account, configured VPC/subnets	ECS project cluster/services/task definitions
GitHub OIDC provider and Actions role	Per-service ECR repositories
Terraform state bucket	Per-service ALB / target group / listener / SGs
Backend control-plane PostgreSQL	Project ECS execution role and runtime secret read policy
GitHub App installation	Managed DB EFS, DB service, Cloud Map namespace/service and DB secret

Destroy invariant

Destroy may remove only resources proven to belong to the project/generation. The workflow does not delete the shared VPC/subnets, OIDC provider, GitHub Actions role or state bucket.

3. GitHub Integration & Repository Admission *[DG-06, DG-07]*

3.1 Authentication and GitHub App responsibilities

Developer login uses a state-validated GitHub OAuth callback. The OAuth state is stored in a short-lived HttpOnly cookie and must match the callback state. A successful login creates a signed session cookie. Repository operations are then mediated through the connected GitHub App installation rather than trusting arbitrary browser-supplied repository content.

3.2 Exact-source principle

The deployment request is resolved to an exact 40-character commit SHA. The reusable workflow checks out that exact commit for deploy actions. Branch names are useful for user selection and source resolution, but the release identity is the immutable commit SHA, not a moving branch reference.

3.3 Reusable workflow pinning

The backend configuration points at an exact 40-character control-plane commit. The backend contract additionally hashes the reusable workflow and the executable files it materializes: release-result builder, release-only task-definition script, runtime verifier, Terraform runtime, fallback builder/contract/template and manifest. A change to any certified executable therefore requires an intentional hash-contract and workflow-pin update.

Current certification branch	Branch head	Pinned control-plane SHA	AWS capability contract
Test 2	54ddc957ca255010b4a0afb60652669623ab22dd	df86ae471140f8fcb8cc893316b64faf8ca37b56	deployguard.railpack-runtime-aws/v10
Test 3	1d5a243a4e3c45c9d9d6d688bd65db2471991ea5	0ebf940d6e3e0e1081f00433c34fccf38d808027	deployguard.railpack-runtime-aws/v13

Why branch head ≠ workflow pin

The final head is a small “pin certified reusable workflow” commit. The workflow pin targets the immediate parent containing the executable defect fixes, so the workflow can be immutable while the branch head records the new reference. The branch copy and pinned copy were verified byte-identical.

4. Build Target, Port & Configuration Contracts *[DG-08, DG-09, DG-10]*

4.1 Deployable-service and build-target resolution

DeployGuard models services explicitly. Each service has a UUID, name, repository-relative directory, build-target revision and runtime configuration revision. Build roots are canonicalized and must remain inside the repository workspace, preventing path traversal or accidental builds from an unrelated directory. The resolver supports standalone JavaScript projects, workspace members/monorepos and Python standalone projects within the certified contracts.

4.2 Port resolution

The service port is not a free-form deployment textbox. DeployGuard attempts to derive one canonical container port from repository evidence with precedence. It inspects framework configuration, entrypoint/listener code, package start commands/Procfile, environment files and framework defaults. Conflicting evidence fails closed instead of choosing an arbitrary port.

Evidence family	Examples	Behavior
Framework/config files	Spring application.properties/YAML, Vite/Nuxt config, package.json config.port	Higher-precedence concrete evidence.
Entrypoint/listener code	Node listen(), Flask/FastAPI/Go listeners, env-with-numeric-fallback patterns	Parses bounded entrypoint files only.
Start command / Procfile	--port 3100, -p 8080	Concrete numeric flags are evidence.
Dynamic platform port	$PORT or ${PORT}	Accepted as valid dynamic usage; ignored as numeric evidence so the framework default/concrete source can resolve.
Framework default	FastAPI/Django 8000; Flask 5000; Next/Nest/Nuxt 3000; Vite 5173; Spring Boot 8080	Used only if earlier evidence is absent.

4.3 Platform-owned HOST/PORT

At runtime DeployGuard injects HOST=0.0.0.0 and PORT=<resolved service port>. User build-environment declarations cannot override these platform-owned runtime keys. The final workflow validates the sealed servicePort and corresponding environment values before materializing Terraform.

4.4 Requirement admission

Required configuration is assembled from explicit repository metadata (for example environment templates, Docker/Compose/config metadata and DeployGuard declarations), platform-owned PORT/HOST and managed-database aliases. The resolver separates build/runtime/both scopes and blocks prohibited overrides. A known boundary remains: arbitrary source-code-only environment access is not exhaustively inferred in every language. Production repositories should declare required variables explicitly rather than relying on static source inference.

4.5 Immutable configuration revisions

Runtime and build configuration carry revision IDs and SHA-256 fingerprints into the dispatch payload. The workflow rejects malformed IDs/fingerprints and secret references that do not match the immutable Secrets Manager ARN/version format. This keeps “same source, different configuration” distinguishable at release time.

5. Build Pipeline: Railpack First, Certified Fallback Second

[DG-11, DG-12]

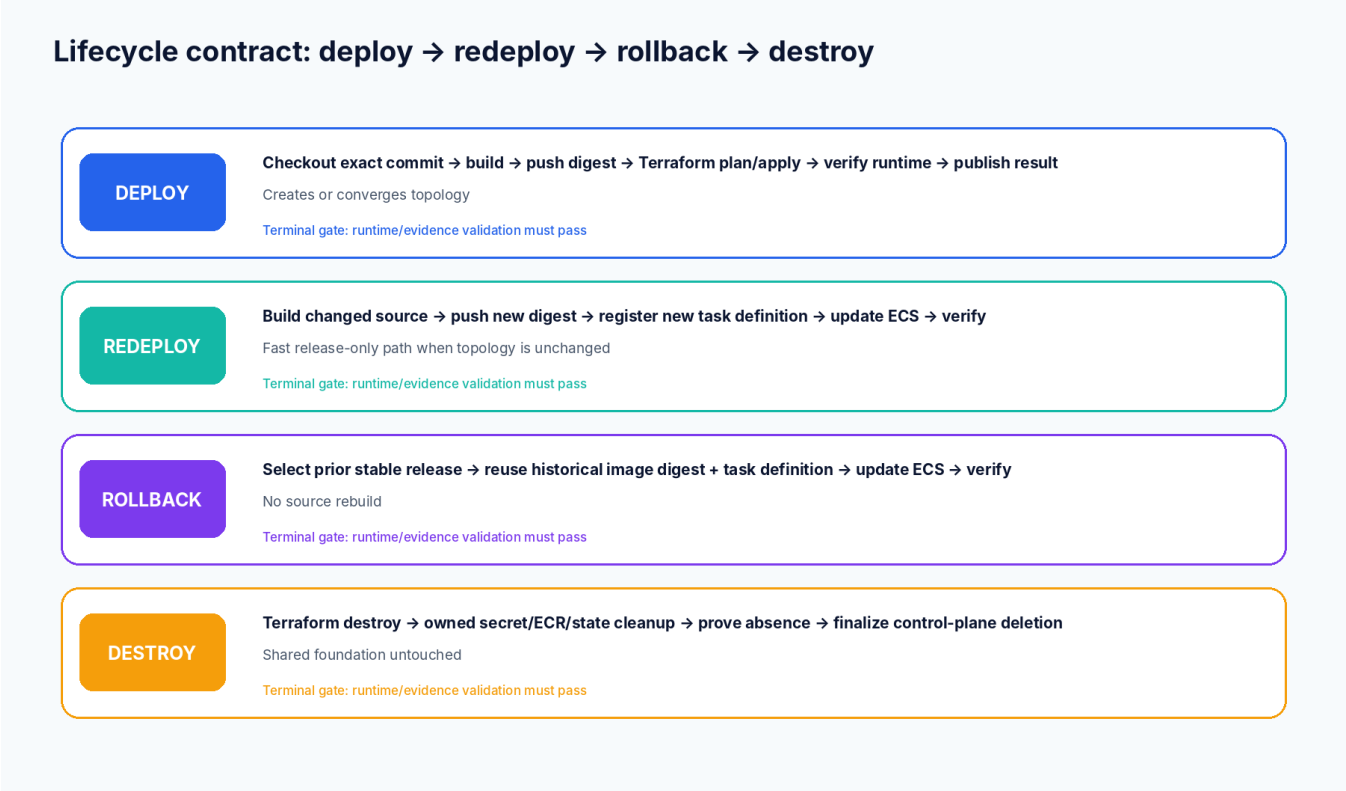


Figure 2. Lifecycle operations and their release semantics.

5.1 Railpack primary builder

The current build path uses Railpack 0.38.0. The workflow verifies the downloaded builder identity/checksum before use, launches a pinned BuildKit daemon and forwards only sealed build capabilities and build-time variables/references. Build output is expected to produce a local immutable image before publication.

5.2 Why the fallback is intentionally narrow

The Docker fallback is not a generic “if build fails, try Docker” escape hatch. A fallback contract first classifies whether the Railpack failure is a proven internal-builder condition. It then selects only an exact certified template/contract from a pinned manifest. Unsupported application contracts fail rather than being silently interpreted by a broad fallback.

5.3 Secret-aware build execution

- **Build secret references:** Fetched from Secrets Manager by immutable version and injected into the build environment without becoming ordinary source files.
- **Fallback BuildKit secrets:** The Docker fallback passes environment/secret material through BuildKit secret mounts rather than embedding values into Docker build arguments wherever possible.
- **Safe evidence:** Build logs are sanitized before being attached to terminal failure events; known secret values and credential-like patterns are redacted.
- **Pinned helper images/actions:** BuildKit and runtime probe images are digest-pinned; external GitHub Actions are pinned to immutable action SHAs.

6. Immutable Image & Release Identity *[DG-11, DG-13]*

6.1 ECR namespace

Each deployable service receives a project/service-scoped ECR repository named from the compact project prefix and compact service prefix. This avoids the earlier full-project-name mismatch and lets IAM simulation match the actual runtime repository pattern.

6.2 Tag then digest

The build tag contains source SHA, operation identity and service identity. After push, the workflow calls ECR to resolve the repository/tag to an immutable sha256 digest and records the release as registry/repository@sha256:.... ECS task definitions use the immutable image reference, so later tag movement cannot change the historical release.

6.3 Release identity chain

Identity	Why it matters
sourceSha	Exact Git commit that supplied application source.
buildTargetRevisionId + fingerprint	Exact interpretation of workspace/build root/package execution.
runtimeConfigRevisionId + fingerprint	Exact runtime/environment/secret binding contract.
operationId	Unique lifecycle attempt and artifact namespace.
ECR imageDigest	Immutable container bytes.
ECS taskDefinitionArn / revision	Immutable runtime container definition used by redeploy/rollback.
AWS runtime verification result	Evidence that the expected identity is actually running and reachable.
terminal release artifact	Structured contract consumed by the backend before promotion.

Release principle
DeployGuard does not define “release = Git commit”. A release is a verified mapping from Git source and configuration to immutable cloud runtime identity.

7. Terraform & AWS Runtime Topology *[DG-14]*

7.1 Per-project/per-service resources

Layer	Current reference resources
Compute	ECS cluster, Fargate task definitions and services, awsvpc networking.
Ingress	Per-service internet-facing ALB, target group and HTTP listener.
Network policy	ALB security group; application security group restricted to ALB on app/probe ports.
Logs	CloudWatch log group per application service plus managed DB log group.
Identity	Per-project ECS task execution role; runtime Secrets Manager read policy only when needed.
Images	Service-scoped ECR repositories managed by the workflow.
State	S3 backend key scoped by project/environment/runtime with native .tflock.

7.2 Platform transport probe

The ALB target group does not depend on an arbitrary application health endpoint. Each task definition includes a small, digest-pinned transport-probe sidecar on a separate high port. It checks whether the application port is reachable inside the task and exposes the platform-owned /_deployguard/transport-ready endpoint to the ALB. This separates “process is accepting TCP” from application-specific business health semantics.

7.3 Current networking boundary

The reference Terraform currently uses the configured public subnets and assigns public IPs to Fargate services. The ALB is internet-facing and the listener is HTTP/80. Security groups limit inbound application traffic to the ALB and managed-database traffic to the attached application service. For a hardened production edition, private application/database subnets plus managed TLS (ACM), HTTPS listeners and optional WAF should be treated as an explicit next security layer rather than implied as already implemented.

No invented TLS claim
The current runtime contract is intentionally described as it exists. HTTPS/WAF capabilities offered by Vercel or by a hardened AWS edge are not credited to DeployGuard unless they are actually automated by the runtime/shared foundation.

8. Runtime Verification & Terminal Evidence *[DG-15, DG-16]*

A successful Terraform apply or ECS update is only an intermediate state. DeployGuard verifies runtime identity and reachability before it emits a releasable result.

8.1 Verification checks

- **ECS service state:** Service exists, expected task definition is active, desired/running state is stable and task identity can be inspected.
- **Immutable image:** Running container image corresponds to the expected immutable digest.
- **Load balancer:** ALB/listener and target group are active; target health reaches the expected state.
- **Public transport:** DNS resolves and the public endpoint/transport path becomes reachable within bounded retries.
- **Runtime bindings:** Service port, configuration revision and secret references are consistent with the admitted release.
- **Managed database:** Database task health, Cloud Map registration and security relationship are verified before the attached application is scaled from zero to one.

8.2 Structured result and failure contracts

Successful workflows publish a deployguard-result-<operationId> artifact containing the release result (currently deployguard.release-result/v5) and AWS runtime verification evidence (deployguard.aws-runtime-verification/v1). Failures produce structured DG_* markers and failure artifacts rather than relying solely on log scraping. The backend validates the artifact's action, operation ID, project/source identity and service-level evidence before it creates or updates the stable release projection.

8.3 Why this matters

This makes GitHub Actions a worker, not the final source of truth. A green workflow without a valid artifact cannot be promoted; conversely, a verified AWS deletion whose database finalization fails can be safely reconciled without repeating destructive cloud operations.

9. Lifecycle Semantics in Detail *[DG-16, DG-17]*

9.1 First deploy / topology change

Deploy materializes the certified Terraform runtime from CONTROL_PLANE_SHA, initializes the S3 backend, validates the configuration, plans and applies project topology, then executes runtime verification. Build artifacts are immutable before Terraform receives them.

9.2 Redeploy fast path

When a LIVE topology remains compatible, redeploy does not perform a full Terraform apply. The workflow builds and publishes the new immutable image, reads existing Terraform outputs, registers a new ECS task-definition revision, updates the existing ECS service and re-runs verification. If the service port/topology would require infrastructure mutation, the release-only contract rejects the path instead of pretending a direct ECS update is safe.

9.3 Rollback

Rollback selects the immediate eligible stable historical release. It reuses the recorded immutable image digest and historical task-definition identity. The rollback script validates that topology and task definition match the current service before update. No historical source checkout or rebuild is required.

9.4 Destroy

Destroy first executes Terraform destruction for project resources. Runtime secrets created outside Terraform are then deleted only after namespace and ownership tags match the immutable project/service/environment input. ECR cleanup accepts a confirmed RepositoryNotFoundException as already clean, but AccessDenied or any unknown failure remains terminal. The workflow deletes and verifies absence of the environment-scoped state object/lock artifacts before emitting project-delete-ready evidence.

Operation	Rebuild?	Terraform apply?	Key release behavior
Deploy	Yes	Yes	Create/converge topology, then prove runtime.
Redeploy	Yes	No (compatible fast path)	New image digest/task revision; same topology.
Rollback	No	No (compatible fast path)	Restore existing historical immutable release.
Destroy	No	Terraform destroy	Remove only owned resources; prove cleanup before project finalization.

10. Managed Database Architecture *[DG-14, DG-18]*

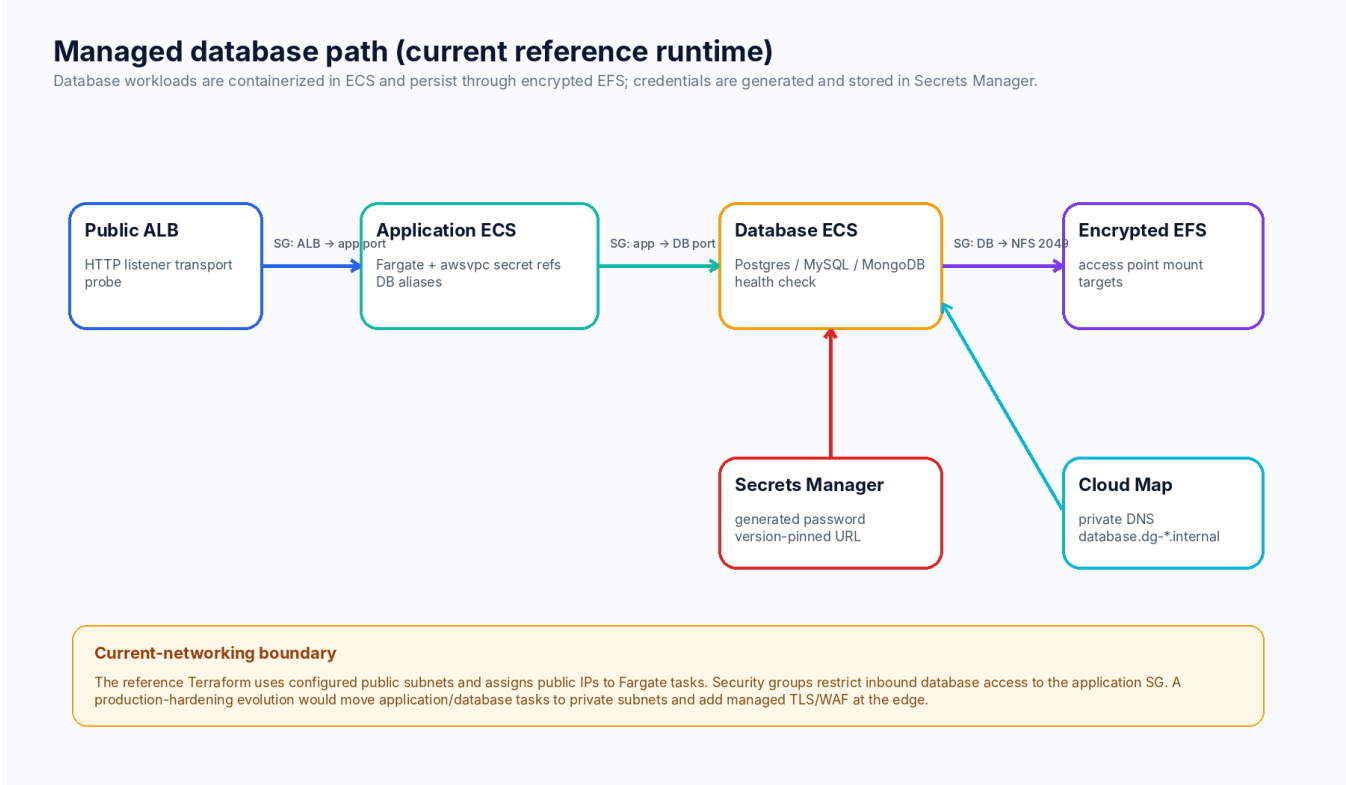


Figure 3. Managed database topology and security boundaries.

10.1 Supported engines

The reference runtime supports PostgreSQL 16, MySQL 8 and MongoDB 8 container images. The managed tier is attached to exactly one application service in the current model. Database port, URL scheme and environment aliases are derived from the selected engine.

10.2 Persistence and credentials

- **Persistence:** Encrypted EFS with a dedicated access point and mount targets; the engine data directory is mounted through the access point.
- **Credentials:** Terraform generates a 32-character password and stores password plus connection URL in an owned Secrets Manager secret.
- **Private service discovery:** Cloud Map creates a private DNS namespace and database service name.
- **Application binding:** Non-secret aliases (host, port, user, database) become environment variables; password/URL aliases become version-pinned ECS secret references.
- **Network policy:** Database SG accepts the engine port only from the attached application SG; EFS SG accepts NFS 2049 only from the database runtime SG.

10.3 Sequenced activation

Terraform intentionally creates a database-attached application service with desired_count=0 while the DB desired count is 1. The runtime verifier waits for database health, Cloud Map registration and expected security-group relationships; only then does it scale the application service to 1. This avoids racing an application startup against an unavailable managed database.

10.4 Reconciliation before a new deployment

The backend reads EFS file systems/access points, the database secret and Terraform state before dispatch to determine whether the control-plane database record still matches AWS reality. This is an important difference from a blind “create DB every time” workflow.

11. Terraform State, Concurrency & Recovery *[DG-05, DG-19]*

11.1 Environment-scoped remote state

Current Test 2/Test 3 capability simulation uses the same path that the workflow uses: projects/<projectId>/<environmentName>/runtime/terraform.tfstate plus the native .tflock. S3 bucket-level List/GetBucketLocation and project-prefix object operations are modeled explicitly. The environment component closes a prior mismatch that could have caused least-privilege simulation to evaluate the wrong object ARN.

11.2 Workflow concurrency

The reusable workflow serializes operations with the GitHub Actions concurrency group deployguard-<projectId> and cancel-in-progress=false. This avoids two workflow runs intentionally racing the same project while preserving the native Terraform S3 lock as the infrastructure-level lock.

11.3 Recovery model

- **Safe retry:** Retry is driven by structured failure diagnostics; unsafe or ambiguous conditions can be rejected rather than replayed automatically.
- **Release finalization recovery:** If AWS release evidence was already verified but database/control-plane finalization failed, the persisted evidence can be revalidated and finalized without rebuilding or changing AWS.
- **Destroy finalization recovery:** If AWS deletion is already verified, project cleanup can complete without redispatching Terraform.
- **Reset fresh:** An explicit project-name confirmation protects fresh-reset semantics and does not bypass compatibility admission.

12. Security Architecture *[DG-03..DG-19]*

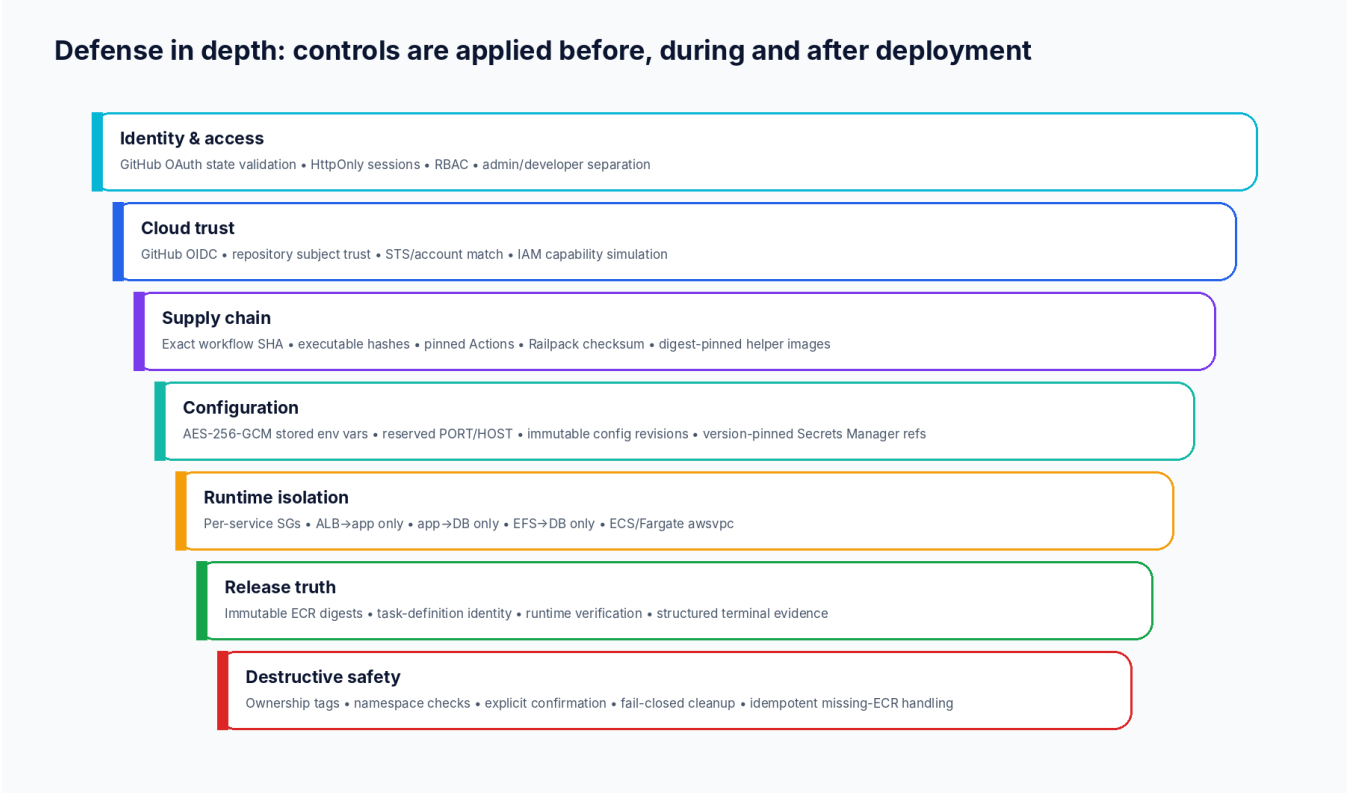


Figure 4. Defense-in-depth layers.

#	Control	How it works
1	OAuth CSRF state	Random state cookie must match GitHub callback state.
2	Secure session cookie	HttpOnly; SameSite=Lax; Secure in production; bounded expiry.
3	Signed session tokens	HMAC-SHA256 signature, explicit audience, issue/expiry timestamps.
4	Admin/developer separation	Different audiences; admin rejects GitHub-linked account path.
5	RBAC	Readonly vs developer/admin; lifecycle mutations require developer/admin.
6	Audit trail	Auth, repository connections and lifecycle requests produce audit events.
7	Environment encryption at rest	AES-256-GCM with random IV and authentication tag; key derived from AUTH_SESSION_SECRET.
8	GitHub Actions OIDC	Actions gets short-lived AWS credentials through web identity, not long-lived AWS keys in workflow secrets.
9	Repository-scoped OIDC trust	Trust policy must contain the GitHub OIDC provider, sts.amazonaws.com audience and authorized repo subject.
10	AWS account consistency	Backend STS account, configured AWS_ACCOUNT_ID and Actions role account must match.
11	Capability simulation	IAM SimulatePrincipalPolicy checks deployment actions against the current exact contract before dispatch.
12	Managed/external role modes	External roles are verified read-only; platform mode may converge only DeployGuard-owned trust/policy pieces.
13	Exact workflow SHA	Reusable workflow reference must be an immutable 40-char commit SHA.
14	Executable hash contract	Workflow + Terraform + runtime scripts + fallback assets are SHA-256 certified by backend code.
15	Pinned third-party actions/images	Checkout, AWS credentials, Terraform, artifact upload, Trivy and helper images are

		immutable-version/digest pinned.
16	Railpack checksum	Primary builder identity/checksum is validated before build.
17	Build-target path containment	Service/build roots must stay inside the repository workspace.
18	Configuration schema validation	UUIDs, SHAs, key formats, RAILPACK capabilities, service ports and secret-reference shapes are validated.
19	Version-pinned Secrets Manager refs	Runtime/build references include immutable secret version IDs; rollback does not follow mutable AWSCURRENT.
20	Secret compensation	Failed provisional changes can delete newly created secret or restore prior AWSCURRENT after ownership checks.
21	Log sanitization	Known secret values and credential-like patterns are redacted before failure evidence / AI evidence.
22	Trivy policy	Optional scan produces a structured security result; enforce mode can block fixable critical application dependency findings.
23	AWS network isolation	ALB→app, app→DB and DB→EFS SG relationships constrain inbound paths.
24	Ownership-bound destroy	Tags/namespaces plus fail-closed ECR/secret/state checks prevent arbitrary resource deletion.

13. Security Trade-offs & Current Hardening Boundary

DeployGuard exposes more cloud controls than a fully managed PaaS, which creates both auditability and responsibility. Several boundaries should be stated explicitly in an academic or production-readiness review.

Current implementation	Security implication	Recommended evolution
ALB listener is HTTP/80	No built-in TLS termination in the current railpack-runtime contract.	ACM certificate + HTTPS/443 listener + HTTP→HTTPS redirect; custom-domain ownership validation.
Application/database Fargate tasks use configured public subnets and public IPs	Inbound is SG-restricted, but tasks are addressable at public-network level.	Private subnets for app/DB, NAT/VPC endpoints for egress, keep ALB public.
Broad AdministratorAccess temporarily used in Test 2 certification	Unblocks certification but violates least-privilege goals.	After E2E, converge back to tested contract-scoped role and bounded backend permissions.
No platform WAF/DDoS control in current runtime contract	Relies on AWS baseline and application controls rather than product-managed L7 policy.	AWS WAF/Shield integration and rate-limit policy if product scope requires edge security.
Requirement inference is metadata-first	A source-only required env var can escape preflight and fail app startup.	Require explicit environment schema or add language-aware static analyzers as optional hints.
Managed DB is self-managed container on ECS/EFS	More ownership, but backups/HA/PITR are not equivalent to RDS/provider-managed database products.	Add backup/PITR/HA strategy or offer RDS-managed tier.

Security positioning

The strongest current DeployGuard controls are release integrity, cloud identity boundaries, immutable secret/release references and fail-closed ownership checks. Edge security, private-subnet topology and managed-database HA/backup depth are separate layers rather than hidden claims.

14. Observability, Diagnostics & Operations *[DG-20, DG-21]*

14.1 Live runtime observability

The observability module is explicitly read-only against the authoritative LIVE generation. It resolves the live runtime then reads CloudWatch logs and metrics, exposes Prometheus-formatted metrics and sanitizes log material. The frontend has dedicated monitoring-presentation tests.

14.2 Failure ownership and diagnostics

Structured DG_* failures are mapped into a failure owner/provider/stage and a retry decision. The product can therefore distinguish an application-startup failure from an IAM/platform defect, builder-internal failure, Terraform failure or runtime-verification failure. This distinction is used by retry and troubleshooting surfaces instead of encouraging blind redeployment.

14.3 AI troubleshooting

AI troubleshooting is an optional module. It assembles project/pipeline/observability evidence through a preprocessor, reuses the log sanitizer and project environment crypto service, and records analysis sessions/messages/results. It should be described as an evidence-assisted diagnosis layer, not an autonomous deployment authority.

14.4 Cost evidence / Infracost

The Projects module includes Infracost integration and GitHub Actions cost-evidence services, while project cost estimate/settings entities provide control-plane storage for cost projections. These functions complement, rather than replace, the AWS bill.

14.5 Notifications

The notification module records preferences/subscriptions/delivery and includes an SNS adapter. Notification material passes through the log sanitizer. Delivery can be enabled or disabled by configuration.

15. Billing, Entitlements, Admin & Terraform Export *[DG-02, DG-22, DG-23]*

15.1 Billing and entitlements

Billing is implemented as a first-class NestJS module with billing account, subscription, usage counter/event, checkout session, invoice, payment and webhook event records. A billing-provider abstraction and entitlement service gate product usage. Repository tests include billing subscription, Stripe billing and billing migration verification. This is a product-layer subscription system; AWS infrastructure charges remain a separate cloud cost surface.

15.2 Administrative and audit surfaces

The application registers a dedicated AdminModule and AuditLogModule. Admin authentication is segregated from developer GitHub login. Lifecycle and auth actions record actor/resource/status/metadata so security and operational changes are explainable after the fact.

15.3 Terraform export

A TerraformExport module/controller/service persists export artifacts associated with projects and infrastructure environments. It provides an exit/inspection surface for generated infrastructure information while the active Railpack runtime remains controlled by the certified workflow.

15.4 Local product packaging

The repository includes local Docker packaging for frontend/backend/PostgreSQL and configuration for Prometheus/Grafana. This local distribution is separate from the application-deployment runtime: DeployGuard itself can be run locally, while customer application releases are executed through GitHub Actions into AWS.

16. Performance & Operational Behavior

No benchmark numbers are invented in this report. The final immutable pins have not yet completed AWS E2E performance measurement. The following statements describe architectural behavior and relative work performed by each lifecycle path.

Path	Expensive work performed	Expected relative behavior	Reason
First deploy	Repository checkout, Railpack build, ECR upload, Terraform plan/apply, ECS/ALB stabilization, verification	Highest latency	Creates topology and waits for multiple AWS control planes.
Changed-source redeploy	Build + ECR push + task-definition registration + ECS rolling update + verification	Lower than full deploy when topology unchanged	Skips Terraform apply and reuses existing infrastructure.
Rollback	No build/push; use historical digest/task definition + ECS update + verification	Potentially fastest recovery path	Reuses immutable artifact rather than rebuilding old source.
Destroy	Terraform destroy + Secrets/ECR/state cleanup + absence verification	Variable	AWS resource deletion and dependency order can dominate.
Managed DB first deploy	Application pipeline plus EFS, DB task, Cloud Map, DB readiness gate	Higher than stateless deploy	Persistence/network/service-discovery readiness must complete before app activation.

16.1 Scaling model

The current Terraform reference creates one application ECS service per deployable service with a fixed task CPU/memory profile (512 CPU units / 1024 MiB in the inspected runtime) and desired count 1 after activation. That is a valid single-instance baseline, not yet an autoscaling contract. AWS Application Auto Scaling dependencies exist in the backend, but autoscaling should not be claimed for the current railpack-runtime until explicit target/policy resources are part of the certified Terraform.

16.2 Where DeployGuard spends complexity

- **More control-plane work than a hosted PaaS:** IAM/OIDC, Terraform state, AWS topology and evidence must be managed explicitly.
- **Less repeated work on redeploy/rollback:** Topology is reused where safe; immutable artifacts allow rollback without source rebuild.
- **More verification before promotion:** ALB/ECS/DNS/transport checks add time deliberately to reduce false “green deploy” states.

17. Verification & Certification Status

Area	Test 2	Test 3	Evidence level
Current head / pin	54ddc957... / df86ae471...	1d5a243a... / 0ebf940d...	GitHub source verified
Branch workflow equals pinned workflow	PASS	PASS	Byte-identical Git blob
Dynamic $PORT defect closure	PASS	PASS	Static contract test
Environment-scoped state simulation	PASS	PASS	Static contract test
ECR destroy idempotency	PASS	PASS	Static executable test
EFS ENI IAM coverage	PASS (v10)	PASS (v13)	Capability contract
Duplicate LB discovery	N/A	PASS / removed	Source inspection
Build / contract suite	PASS	PASS	Reported local test execution
Newest-pin AWS E2E	UNVERIFIED / blocked before dispatch	UNVERIFIED / separate AWS config missing	No deployment dispatched

17.1 Historical Test 2 live evidence (older pin)

Lifecycle event	Historical result	GitHub run
Normal deploy	PASS	35130842011
Normal rollback	PASS	35132897748
Normal destroy	PASS	35133429298
Changed-source redeploy	Not proven on that run set	Workflow ran but source SHA had not changed
Managed DB deployment	Reached workflow/build; application did not start	Test application startup command/config issue; not evidence of an AWS permission failure

Release gate
Do not label the newest pins “production-certified” until the exact frozen Test 2 and Test 3 environments execute the required normal + managed-DB lifecycles. The source is frozen specifically so any new failure can be attributed to runtime/configuration rather than moving code.

18. Competitive Architecture: DeployGuard vs Vercel, Render & Heroku *[V-01..V-05, R-01..R-05, H-01..H-04]*

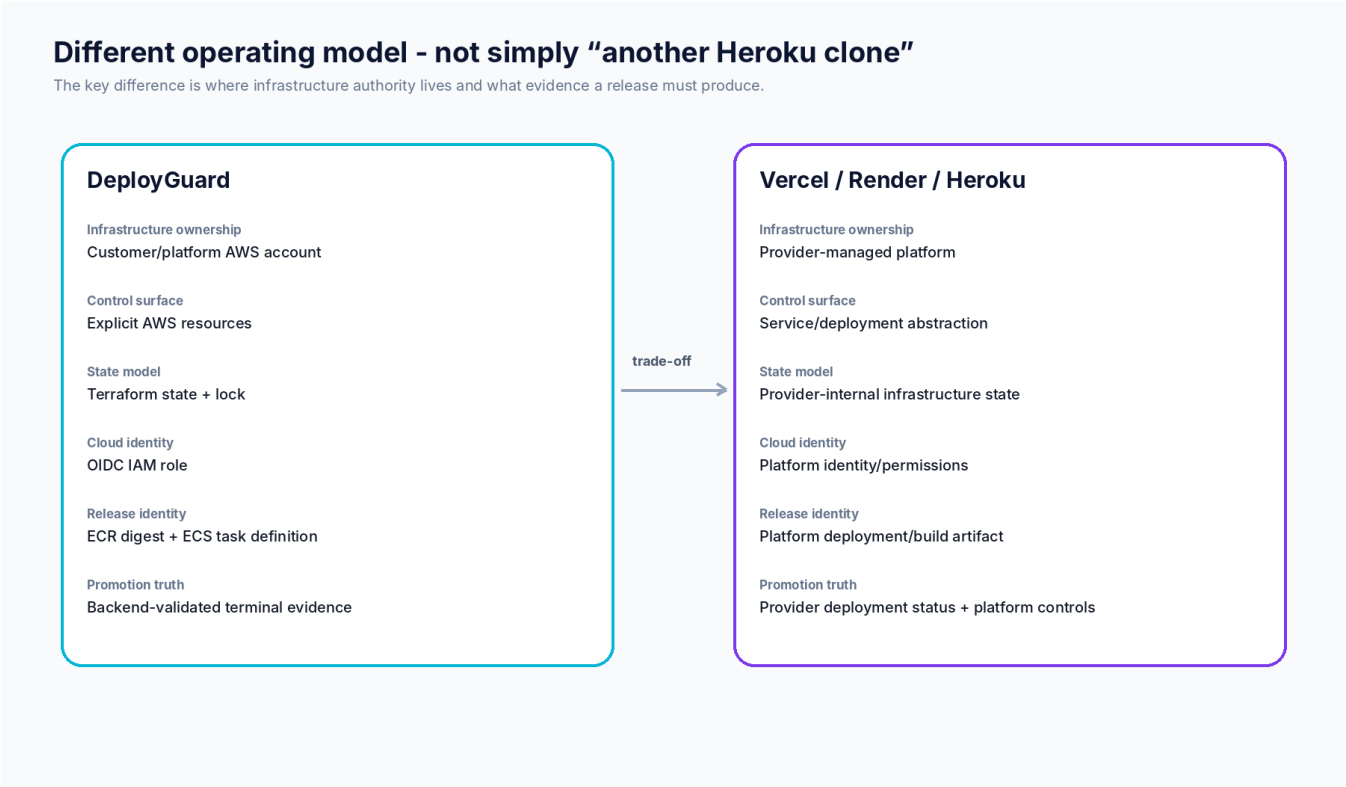


Figure 5. Fundamental operating-model difference.

The fairest comparison is architectural, not a claim that one product is universally “better”. Vercel, Render and Heroku intentionally hide more infrastructure and provide strong managed-platform capabilities. DeployGuard intentionally exposes the target AWS account and makes AWS identity, Terraform state and release evidence part of its contract. That gives control and auditability at the price of operational complexity.

19. Side-by-Side Contract Matrix

Dimension	DeployGuard	Vercel	Render	Heroku
Primary abstraction	Verified release into customer/platform AWS	Web deployment / framework primitives	Managed service / deploy	App release / dyno formation
Infrastructure ownership	Target AWS account owns ECS/ECR/ALB/EFS/Secrets/ Cloud Map	Vercel-managed infrastructure	Render-managed infrastructure	Heroku-managed runtime
Build model	Railpack 0.38 + narrow certified Docker fallback	Framework build + Build Output API / Vercel builders	Native runtime build commands or Docker	Buildpacks by default; Docker/OCI supported
Cloud identity	GitHub OIDC → explicit AWS IAM role	Platform handles underlying cloud identity	Platform handles underlying cloud identity	Platform handles underlying runtime identity
IaC/state	Terraform S3 state + .tflock visible to operator	Platform model; Build Output API describes deployment output	Blueprint YAML for Render resources	Platform release/app model; no customer Terraform state for dynos
Release identity	Source + config/build revisions + ECR digest + ECS task definition + evidence	Deployment ID/URL/build output	Deploy + retained build artifact	Release version + slug/OCI image + config vars
Redeploy	Direct ECS release path when topology unchanged	Redeploy re-runs build for commit/config	New deploy from linked source/image	New release via deploy/build
Rollback	Reuse historical digest/task definition; verify runtime	Instant rollback points domains to eligible existing deployment	New deploy using retained target build artifact; target env values reused	New release copying prior slug/OCI + config vars
Persistence	App task ephemeral; DB persistence via encrypted EFS	Functions read-only filesystem + /tmp scratch; use data services	Ephemeral by default; optional persistent disk or managed Postgres/KV	Dyno filesystem ephemeral; use add-ons/external data
Managed DB model	Container DB on ECS + EFS + Secrets + Cloud Map	Typically external/marketplace data sources near Functions	Managed Render Postgres/KV plus custom DB on disk	Data add-ons / external state
Promotion gate	Backend validates structured AWS runtime evidence	Platform deployment/promotion status	Platform deploy health/service state	Platform release/dyno health model
Security emphasis	IAM/OIDC, immutable contract, ownership-bound cloud operations	Managed edge security, deployment protection, WAF, DDoS, RBAC	Managed service isolation, private network, platform controls	Dyno isolation, platform runtime controls, Private Spaces/Shield tiers
Operational trade-off	Maximum AWS visibility/control; higher setup responsibility	Very low infra burden for web/Functions workloads	Balanced managed services + container flexibility	Mature application-centric PaaS abstraction

20. DeployGuard vs Vercel *[V-01..V-05]*

20.1 What Vercel optimizes

Vercel defines a deployment as the result of a successful project build and can create deployments through Git, CLI, Deploy Hooks or REST API. Its Build Output API maps framework output to Vercel primitives such as static assets, Functions, routing and caching. Vercel Functions automatically scale inside Vercel-managed infrastructure, and Vercel supplies platform-level firewall/DDoS/deployment-protection features. [V-01, V-02, V-04]

20.2 Contract difference

- **DeployGuard:** release promotion depends on an explicit AWS identity chain, Terraform/ECS runtime and backend-validated evidence.
- **Vercel:** deployment/promotion is a Vercel platform operation; the customer does not manage underlying ECS/IAM/Terraform resources.
- **DeployGuard rollback:** select prior immutable ECR/task definition, update ECS, verify.
- **Vercel Instant Rollback:** reassigns production domains to an eligible existing deployment instead of rebuilding it; environment/external-data caveats remain. [V-03]
- **Security difference:** Vercel offers mature managed edge/WAF/DDoS/deployment protection. DeployGuard's strongest differentiated security today is cloud identity/release integrity rather than an equivalent global edge security layer. [V-04]

20.3 Best-fit difference

Vercel is particularly strong for frontend/framework-centric workloads and serverless Functions where infrastructure abstraction is desirable. DeployGuard is better described as an AWS deployment-control product for teams that want to see and own the underlying cloud resources and release evidence. These are overlapping but not identical jobs.

21. DeployGuard vs Render *[R-01..R-05]*

21.1 Similarities

Render and DeployGuard both support repository-driven services, configurable start commands, container-style runtimes, deploy history, rollback and persistent-data options. Render additionally exposes an IaC model through render.yaml Blueprints. [R-01, R-04]

21.2 Key differences

Topic	DeployGuard	Render
Runtime ownership	ECS/ALB/ECR resources live in target AWS account and are visible/operable there.	Service instances are Render-managed and exposed through Render's service abstraction.
Rollback	Historical AWS image/task identity, then runtime verification.	Creates a rollback deploy using the target retained build artifact; target deploy env vars/start command/health path are reused where documented. [R-02]
Persistence	Managed DB persistence is explicit EFS + container DB topology.	Services are ephemeral by default; persistent disk can be attached, with single-instance/no zero-downtime limitations; Render also provides managed Postgres/KV. [R-03, R-05]
IaC	Terraform against AWS resources + visible S3 state.	Blueprint YAML is source of truth for Render services/databases/env groups. [R-04]
Private networking	Cloud Map + SG relationships for managed DB; current app topology uses public subnets.	Private services are reachable only from other Render services on the same private network. [R-05]

22. DeployGuard vs Heroku *[H-01..H-04]*

22.1 Heroku release model

Heroku runs applications on Heroku-managed dynos. A release is created when code, config vars or add-on resources change. Heroku's rollback command creates a new release that copies the target release's compiled slug or OCI image and config vars; it does not roll back external/add-on data state. [H-01, H-02]

22.2 Key differences

- **Compute abstraction:** Heroku exposes dyno types/formations; DeployGuard exposes AWS ECS/Fargate services and task definitions.
- **Filesystem:** Heroku dynos are ephemeral; DeployGuard application tasks are likewise not a persistence layer, while the managed DB contract explicitly mounts EFS. [H-01]
- **Configuration:** Heroku config vars are platform-managed environment configuration and changes create releases. DeployGuard stores project variables encrypted in the control plane and materializes immutable Secrets Manager-backed runtime references for secrets. [H-03]
- **Container deployment:** Heroku supports Container Registry/Runtime in addition to default buildpacks. DeployGuard always turns the admitted service into an image for ECR/ECS, using Railpack first and certified fallback only in bounded conditions. [H-04]
- **Operational model:** Heroku removes nearly all cloud-IAM/Terraform responsibility; DeployGuard makes those controls visible and testable.

23. The DeployGuard Contract - What Is Actually Different

Contract question	DeployGuard answer
What source did we deploy?	An exact Git SHA, sealed into dispatch and terminal evidence.
How was the repository interpreted?	Immutable build-target revision/fingerprint plus service directory and execution contract.
What configuration did it use?	Immutable runtime config revision/fingerprint; platform-owned HOST/PORT; version-pinned secret references.
Who was allowed to touch AWS?	A repository-authorized GitHub OIDC execution role whose effective actions are simulated before dispatch.
Which AWS account?	Backend STS account, configured account and Actions role account must be identical.
What bytes are running?	ECR sha256 digest, not a mutable tag.
What runtime definition?	Specific ECS task-definition ARN/revision.
Did infrastructure apply?	Terraform result against environment-scoped S3 state and native lockfile.
Is it really serving?	ECS/ALB/target/DNS/public transport/runtime binding checks.
Can we roll back without rebuilding?	Yes, when the immediate prior stable immutable release is eligible.
Can destroy delete arbitrary AWS resources?	No; resource names/tags/namespaces and shared-platform exclusions bound deletion.
Who decides release success?	Backend after validating terminal evidence - not a green GitHub job alone.

Positioning sentence

DeployGuard is an evidence-driven AWS PaaS control plane: it trades some managed-platform simplicity for explicit cloud ownership, immutable release identity and auditable deployment contracts.

24. Current Limitations & Hardening Roadmap

Priority	Work item	Why
P0	Complete E2E freeze certification	Run the exact frozen Test 2 and Test 3 normal + managed-DB lifecycles; record run IDs, account, digests, task revisions, health and cleanup evidence.
P0	Credential/session runbook	Make local backend identity loading and GitHub authenticated test-session setup deterministic so certification cannot be blocked before dispatch.
P1	HTTPS/custom domains	ACM certificate lifecycle, HTTPS listener, redirect, domain validation and Route 53/custom-domain contract.
P1	Private runtime subnets	Keep ALB public; move application/database tasks to private subnets with controlled egress/VPC endpoints.
P1	Least privilege after certification	Replace temporary AdministratorAccess with contract-generated/tested Actions policy and explicit backend policy.
P1	Managed DB durability	Backups, restore/PITR, resource sizing, HA/replication choices; or introduce an RDS tier.
P2	Autoscaling	Certified Application Auto Scaling target/policy contract and UI controls; verify scaling evidence.
P2	Edge security	AWS WAF/rate limiting/Shield integration if DeployGuard is intended to compete on managed edge protection.
P2	Configuration schemas	Optional deployguard.yml / env schema to make required variables, health semantics and processes completely explicit.

25. Recommended FYP Demo & Presentation Sequence

9. Open the DeployGuard dashboard and show GitHub-authenticated project/repository selection.
10. Show detected service/build target, resolved port, environment/secret scopes and optional managed database attachment.
11. Explain the pre-dispatch gates: exact source SHA, AWS account match, OIDC trust and IAM capability simulation.
12. Start a normal Deploy and narrate source checkout → Railpack → ECR digest → Terraform → ECS/ALB → runtime verification → terminal evidence.
13. Change visible application source and Redeploy; show changed source SHA, new ECR digest and new task-definition revision without a full topology rebuild.
14. Rollback; show the previous immutable digest/task definition returning without rebuilding source.
15. For managed DB, show the EFS/Secrets/Cloud Map topology and the “DB first, app second” activation gate.
16. Destroy; show project resources removed while shared VPC/state bucket/OIDC role remain.
17. Finish with the competitive slide: DeployGuard owns AWS evidence/contracts; Vercel/Render/Heroku intentionally own more infrastructure for the user.

Demo discipline

Use exact run IDs and actual runtime evidence on presentation day. If Test 3 remains externally blocked, say “source-certified, environment not yet live-certified” instead of hiding the boundary.

Appendix A. Contract & Failure Glossary

Term	Meaning
CONTROL_PLANE_SHA	Exact DeployGuard commit from which workflow executable runtime files are fetched.
sourceSha	Exact application Git commit.
operationId	Unique deployment lifecycle operation UUID.
runtimeConfigRevisionId	Immutable control-plane revision identifying runtime config.
buildTargetRevisionId	Immutable control-plane revision identifying build interpretation.
release-only	Direct ECS release path used when topology is already LIVE and compatible.
DG_* failure	Structured DeployGuard diagnostic code emitted with stage/service evidence.
stable release	Backend-validated release record eligible as current or immediate rollback target.
generation	Deployment generation used to bind runtime resources and revisions.
managed DB	ECS-hosted Postgres/MySQL/MongoDB tier with EFS persistence and Secrets Manager credentials.
transport probe	Platform sidecar that proves the application port is accepting connections independent of business health semantics.

Appendix B. Source References

DeployGuard repository references

ID	Path	Used for
DG-01	backend/src/app.module.ts	Product modules/entities and global auth middleware.
DG-02	backend/src/projects/projects.module.ts + projects.controller.ts	Projects, services, GitHub connectivity, lifecycle, state, DB tier and supporting services.
DG-03	backend/src/auth/auth.controller.ts	GitHub OAuth state, session cookie and audit events.
DG-04	backend/src/auth/auth.service.ts	PBKDF2 password hashing, HMAC session token audiences/expiry, GitHub OAuth handling.
DG-05	backend/src/projects/railpack-deployment.service.ts	Lifecycle admission, dispatch, retry/recovery, rollback/destroy finalization.
DG-06	backend/src/projects/github-app.service.ts	GitHub App/caller workflow integration.
DG-07	backend/src/projects/github-actions-workflow-contract.ts	Exact workflow pin and executable SHA-256 compatibility contract.
DG-08	backend/src/projects/build-target-resolver.service.ts	Build-target admission.
DG-09	backend/src/projects/service-port-resolver.ts	Port-evidence precedence and dynamic PORT behavior.
DG-10	backend/src/projects/deployment-requirement-resolver.service.ts	Environment/managed-DB requirement admission.
DG-11	.github/workflows/deployguard-reusable.yml	Build, OIDC, Terraform lifecycle, ECR, Trivy, artifact publication and cleanup.
DG-12	infrastructure/docker-fallback/*	Certified fallback classification/template/manifest.
DG-13	infrastructure/railpack-runtime/register-release-task-definitions.sh	Direct ECS redeploy/rollback boundary.
DG-14	infrastructure/railpack-runtime/main.tf	ECS/ALB/SG/Secrets/EFS/Cloud Map runtime.
DG-15	infrastructure/railpack-runtime/verify-runtime.sh	Runtime/DB/ALB/DNS/transport verification.
DG-16	infrastructure/railpack-runtime/build-release-result.sh	Terminal release evidence contract.
DG-17	backend/src/projects/github-actions-runtime-secret.service.ts	Immutable runtime-secret materialization and compensation.
DG-18	backend/src/projects/managed-database-reconciliation.service.ts	Backend reconciliation of AWS DB/EFS/secret/state.
DG-19	backend/src/projects/github-actions-aws-capability-contract.ts + service.ts	IAM capability contract, simulation and policy reconciliation.
DG-20	backend/src/observability/observability.module.ts	CloudWatch/Prometheus live-runtime observability.
DG-21	backend/src/ai-troubleshooting/ai-troubleshooting.module.ts	Evidence-assisted AI troubleshooting.
DG-22	backend/src/billing/billing.module.ts	Subscription/usage/checkout/invoice/payment/webhook and entitlements.
DG-23	backend/src/terraform-export/terraform-export.module.ts	Terraform export artifact surface.
DG-24	backend/src/projects/project-environment-crypto.service.ts	AES-256-GCM encryption for stored project environment values.

Official competitor documentation

ID	Official source	URL
V-01	Vercel - Deploying to Vercel	https://vercel.com/docs/deployments/overview
V-02	Vercel - Build Output API	https://vercel.com/docs/build-output-api
V-03	Vercel - Instant Rollback / Promoting Deployments	https://vercel.com/docs/instant-rollback

V-04	Vercel - Security / Deployment Protection	https://vercel.com/docs/security
V-05	Vercel - Functions / runtimes	https://vercel.com/docs/functions
R-01	Render - Deploying on Render	https://render.com/docs/deploys
R-02	Render - Rollbacks	https://render.com/docs/rollbacks
R-03	Render - Persistent Disks	https://render.com/docs/disks
R-04	Render - Blueprints (IaC)	https://render.com/docs/infrastructure-as-code
R-05	Render - Service Types / Private Services	https://render.com/docs/service-types
H-01	Heroku - Dynos	https://devcenter.heroku.com/articles/dynos
H-02	Heroku - Releases	https://devcenter.heroku.com/articles/releases
H-03	Heroku - Config Vars	https://devcenter.heroku.com/articles/config-vars
H-04	Heroku - Container Registry & Runtime	https://devcenter.heroku.com/articles/container-registry-and-runtime

Method note

Competitor comparisons were checked against official vendor documentation current to September 2026. Product capabilities change over time; this report compares documented architecture and behavior rather than marketing claims. DeployGuard claims are grounded in the frozen Test 2/Test 3 source and explicitly separated from newest-pin AWS E2E status.